# Export Kode Proyek ABSA - Kelompok 1

PDF ini berisi kode notebook EDA dan aplikasi Streamlit dataset explorer yang sudah disesuaikan dengan dataset terbaru.

## Notebook: Kelp1_EDA.ipynb

Exploratory Data Analysis (EDA) - Kelompok 1

## Kode notebook cell 1

```
from pathlib import Path
import json
import re
from collections import Counter, defaultdict

import pandas as pd
import matplotlib.pyplot as plt

pd.set_option("display.max_colwidth", 120)

DATA_DIR = Path("../dataset")
RAW_PATH = DATA_DIR / "Kelp1_dataset_1.csv"
CLEAN_PATH = DATA_DIR / "Kelp1_dataset_2.csv"
ANNOTATION_PATH = DATA_DIR / "db_nlp1_genap2526.jsonl"
TEXTCAT_IRR_PATH = DATA_DIR / "nlp1_iaaa_textcat.jsonl"
NER_IRR_PATH = DATA_DIR / "nlp1_iaaa.jsonl"
```

1. Fungsi bantu

## Kode notebook cell 2

```
LABELS = [
    "PRODUCT_POSITIVE", "PRODUCT_NEGATIVE", "PRODUCT_NEUTRAL",
    "PRICE_POSITIVE", "PRICE_NEGATIVE", "PRICE_NEUTRAL",
    "PLACE_POSITIVE", "PLACE_NEGATIVE", "PLACE_NEUTRAL",
    "PROMOTION_POSITIVE", "PROMOTION_NEGATIVE", "PROMOTION_NEUTRAL",
    "OUT_OF_TOPIC",
]

ASPEK_ID = {"PRODUCT": "Produk", "PRICE": "Harga", "PLACE": "Tempat", "PROMOTION": "Promosi", "OUT": "Di luar to
pik"}
SENTIMEN_ID = {"POSITIVE": "Positif", "NEGATIVE": "Negatif", "NEUTRAL": "Netral", "TOPIC": "Di luar topik"}

def norm_text(value):
    return re.sub(r"\s+", " ", str(value or "").strip())

def hitung_kata(text):
    return len(re.findall(r"\b\w+\b", str(text or "")))

def label_indonesia(label):
    label = str(label)
    if label == "OUT_OF_TOPIC":
        return "Di luar topik"
    parts = label.split("_")
    if len(parts) >= 2:
        return f"{ASPEK_ID.get(parts[0], parts[0].title())} - {SENTIMEN_ID.get(parts[1], parts[1].title())}"
    return label

def aspek_label(label):
    label = str(label)
    if label == "OUT_OF_TOPIC":
        return "Di luar topik"
    return ASPEK_ID.get(label.split("_")[0], label.split("_")[0].title())

def sentimen_label(label):
    label = str(label)
    if label == "OUT_OF_TOPIC":
        return "Di luar topik"
    parts = label.split("_")
    return SENTIMEN_ID.get(parts[1], parts[1].title()) if len(parts) > 1 else ""

def load_jsonl(path):
    rows = []
    with open(path, "r", encoding="utf-8-sig") as f:
        for line in f:
            line = line.strip()
            if line:
                rows.append(json.loads(line))
    return rows

def load_json_or_jsonl(path):
    text = Path(path).read_text(encoding="utf-8-sig").strip()
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        return load_jsonl(path)
```

2. Load dataset mentah, preprocessing, dan anotasi

## Kode notebook cell 3

```
raw_df = pd.read_csv(RAW_PATH)
```

```
clean_df = pd.read_csv(CLEAN_PATH)
annotation_rows = load_jsonl(ANNOTATION_PATH)

print("Jumlah baris dataset mentah:", len(raw_df))
print("Jumlah baris dataset preprocessing:", len(clean_df))
print("Jumlah baris anotasi:", len(annotation_rows))
print("Jumlah review unik pada anotasi:", len({norm_text(r.get("text", "")) for r in annotation_rows}))
print("Jumlah annotator:", len({r.get("_annotator_id") for r in annotation_rows if r.get("_annotator_id")}))

raw_df.head()
```

3. Ringkasan preprocessing sederhana

## Kode notebook cell 4

```
preprocessing_summary = pd.DataFrame({
    "Tahap": ["Dataset mentah", "Dataset preprocessing"],
    "Jumlah baris": [len(raw_df), len(clean_df)],
    "Jumlah teks tidak kosong": [raw_df["text"].notna().sum() if "text" in raw_df.columns else 0, clean_df["text
    "].notna().sum() if "text" in clean_df.columns else 0],
    "Jumlah teks unik": [raw_df["text"].map(norm_text).nunique() if "text" in raw_df.columns else 0, clean_df["t
    ext"].map(norm_text).nunique() if "text" in clean_df.columns else 0],
})
preprocessing_summary
```

4. Agregasi anotasi

## Kode notebook cell 5

```
raw_df["text_norm"] = raw_df.get("text", pd.Series(dtype=str)).map(norm_text)
clean_df["text_norm"] = clean_df["text"].map(norm_text)

raw_meta = (raw_df.dropna(subset=["text"])
            .drop_duplicates("text_norm")
            .set_index("text_norm")[["title", "stars"]]
            .to_dict("index"))
clean_meta = (clean_df.drop_duplicates("text_norm")
              .set_index("text_norm")[["title"]]
              .to_dict("index"))

by_text = defaultdict(list)
for row in annotation_rows:
    by_text[norm_text(row.get("text", ""))].append(row)

review_rows = []
label_rows = []
entity_rows = []

for review_id, (text_norm, rows) in enumerate(by_text.items(), start=1):
    text = rows[0].get("text", text_norm)
    annotators = sorted({r.get("_annotator_id", "") for r in rows if r.get("_annotator_id")})
    n_annotators = len(annotators) if annotators else len(rows)
    min_votes = 1 if n_annotators <= 1 else (n_annotators // 2 + 1)

    label_counts = Counter()
    for r in rows:
        for lab in r.get("accept") or []:
            label_counts[lab] += 1

    final_labels = [lab for lab in LABELS if label_counts.get(lab, 0) >= min_votes]
    if not final_labels and label_counts:
        max_vote = max(label_counts.values())
        final_labels = [lab for lab in LABELS if label_counts.get(lab, 0) == max_vote]

    meta = dict(clean_meta.get(text_norm, {}))
    meta.update(raw_meta.get(text_norm, {}))

    review_rows.append({
        "review_id": review_id,
        "category": "Kuliner",
        "business_name": meta.get("title", ""),
        "rating": meta.get("stars", ""),
        "review_text": text,
        "word_count": hitung_kata(text),
        "token_count": len(rows[0].get("tokens") or []),
        "n_annotations": len(rows),
        "n_annotators": n_annotators,
        "final_labels": final_labels,
        "final_labels_id": ", ".join(label_indonesia(l) for l in final_labels),
    })

    for lab in final_labels:
        label_rows.append({
            "review_id": review_id,
            "kode_label": lab,
            "label": label_indonesia(lab),
            "aspek": aspek_label(lab),
            "sentimen": sentimen_label(lab),
            "jumlah_annotator_setuju": label_counts.get(lab, 0),
            "jumlah_annotator": n_annotators,
        })

    for r in rows:
        for sp in r.get("spans") or []:
            start, end = sp.get("start"), sp.get("end")
            entitas = text[start:end] if isinstance(start, int) and isinstance(end, int) else sp.get("text", "")
            lab = sp.get("label", "")
```

```
            entity_rows.append({
                "review_id": review_id,
                "annotator_id": r.get("_annotator_id", ""),
                "entitas": norm_text(entitas),
                "kode_label": lab,
                "label": label_indonesia(lab),
                "aspek": aspek_label(lab),
                "sentimen": sentimen_label(lab),
            })

reviews_df = pd.DataFrame(review_rows)
labels_df = pd.DataFrame(label_rows)
entities_df = pd.DataFrame(entity_rows)

print("Jumlah review hasil agregasi:", len(reviews_df))
print("Jumlah label ABSA final:", len(labels_df))
print("Jumlah entitas NER/span:", len(entities_df))

reviews_df.head()
```

5. Distribusi label ABSA

## Kode notebook cell 6

```
label_dist = labels_df["label"].value_counts().reset_index()
label_dist.columns = ["Label", "Jumlah"]
label_dist
```

## Kode notebook cell 7

```
plt.figure(figsize=(10, 5))
plt.bar(label_dist["Label"], label_dist["Jumlah"])
plt.xticks(rotation=90)
plt.xlabel("Label")
plt.ylabel("Jumlah")
plt.title("Distribusi Label ABSA Final")
plt.tight_layout()
plt.show()
```

6. Distribusi aspek dan sentimen

## Kode notebook cell 8

```
aspect_dist = labels_df["aspek"].value_counts().reset_index()
aspect_dist.columns = ["Aspek", "Jumlah"]
aspect_dist
```

## Kode notebook cell 9

```
plt.figure(figsize=(7, 4))
plt.bar(aspect_dist["Aspek"], aspect_dist["Jumlah"])
plt.xlabel("Aspek")
plt.ylabel("Jumlah")
plt.title("Distribusi Data per Aspek")
plt.tight_layout()
plt.show()

sentiment_dist = labels_df["sentimen"].value_counts().reset_index()
sentiment_dist.columns = ["Sentimen", "Jumlah"]
sentiment_dist
```

7. Distribusi entitas NER / span ABSA

## Kode notebook cell 10

```
entity_label_dist = entities_df["label"].value_counts().reset_index()
entity_label_dist.columns = ["Label Entitas", "Jumlah"]
entity_label_dist.head(20)
```

## Kode notebook cell 11

```
plt.figure(figsize=(10, 5))
plt.bar(entity_label_dist["Label Entitas"], entity_label_dist["Jumlah"])
plt.xticks(rotation=90)
plt.xlabel("Label Entitas")
plt.ylabel("Jumlah")
plt.title("Distribusi Entitas/Span ABSA")
plt.tight_layout()
plt.show()
```

8. Distribusi panjang review

## Kode notebook cell 12

```
reviews_df[["word_count", "token_count"]].describe()
```

## Kode notebook cell 13

```
plt.figure(figsize=(9, 4))
plt.hist(reviews_df["word_count"], bins=30)
plt.xlabel("Jumlah kata")
```

```
plt.ylabel("Jumlah review")
plt.title("Distribusi Panjang Review Berdasarkan Jumlah Kata")
plt.tight_layout()
plt.show()

plt.figure(figsize=(9, 4))
plt.hist(reviews_df["token_count"], bins=30)
plt.xlabel("Jumlah token")
plt.ylabel("Jumlah review")
plt.title("Distribusi Panjang Review Berdasarkan Jumlah Token")
plt.tight_layout()
plt.show()
```

9. Matriks korelasi antar label

## Kode notebook cell 14

```
label_matrix = pd.crosstab(labels_df["review_id"], labels_df["kode_label"])
label_matrix = label_matrix.reindex(columns=[l for l in LABELS if l in label_matrix.columns], fill_value=0)
label_corr = label_matrix.corr()
label_corr_display = label_corr.copy()
label_corr_display.index = [label_indonesia(i) for i in label_corr_display.index]
label_corr_display.columns = [label_indonesia(c) for c in label_corr_display.columns]
label_corr_display
```

## Kode notebook cell 15

```
plt.figure(figsize=(10, 8))
plt.imshow(label_corr.values, aspect="auto")
plt.xticks(range(len(label_corr.columns)), [label_indonesia(c) for c in label_corr.columns], rotation=90)
plt.yticks(range(len(label_corr.index)), [label_indonesia(i) for i in label_corr.index])
plt.colorbar()
plt.title("Matriks Korelasi Antar Label ABSA")
plt.tight_layout()
plt.show()
```

10. Inter-Annotator Agreement (IRR)

## Kode notebook cell 16

```
textcat_irr = load_json_or_jsonl(TEXTCAT_IRR_PATH)
ner_irr = load_json_or_jsonl(NER_IRR_PATH)

textcat_irr_df = pd.DataFrame([
    {"kode_label": label, "label": label_indonesia(label), **metrics}
    for label, metrics in textcat_irr.items()
])
textcat_irr_df.head(13)
```

## Kode notebook cell 17

```
ner_summary = pd.DataFrame([
    {"metrik": key, "nilai": value}
    for key, value in ner_irr.items()
    if not isinstance(value, (list, dict))
])
ner_summary
```

11. Kesimpulan singkat

## Streamlit: Kelp1_app.py

```
from pathlib import Path
import json
import re
from collections import Counter, defaultdict

import pandas as pd
import streamlit as st
import matplotlib.pyplot as plt

st.set_page_config(page_title="Explorer Dataset Kelp1", layout="wide")

BASE_DIR = Path(__file__).resolve().parents[1]
DATA_DIR = BASE_DIR / "dataset"

LABELS = [
    "PRODUCT_POSITIVE", "PRODUCT_NEGATIVE", "PRODUCT_NEUTRAL",
    "PRICE_POSITIVE", "PRICE_NEGATIVE", "PRICE_NEUTRAL",
    "PLACE_POSITIVE", "PLACE_NEGATIVE", "PLACE_NEUTRAL",
    "PROMOTION_POSITIVE", "PROMOTION_NEGATIVE", "PROMOTION_NEUTRAL",
    "OUT_OF_TOPIC",
]
ASPEK_ID = {
    "PRODUCT": "Produk",
    "PRICE": "Harga",
    "PLACE": "Tempat",
    "PROMOTION": "Promosi",
    "OUT": "Di luar topik",
}
SENTIMEN_ID = {
    "POSITIVE": "Positif",
    "NEGATIVE": "Negatif",
    "NEUTRAL": "Netral",
    "TOPIC": "Di luar topik",
}
KOLOM_ID = {
    "review_id": "ID Ulasan",
    "category": "Kategori Usaha",
    "business_name": "Nama Usaha",
    "rating": "Rating",
    "review_text": "Teks Ulasan",
    "clean_text": "Teks Bersih",
    "word_count": "Jumlah Kata",
    "token_count": "Jumlah Token",
    "n_annotations": "Jumlah Anotasi",
    "n_annotators": "Jumlah Annotator",
    "final_labels_id": "Label ABSA Final",
    "kode_label": "Kode Label",
    "label": "Label",
    "aspek": "Aspek",
    "sentimen": "Sentimen",
    "jumlah_annotator_setuju": "Annotator Setuju",
    "jumlah_annotator": "Jumlah Annotator",
    "annotator_id": "ID Annotator",
    "entitas": "Entitas",
    "text": "Teks Ulasan",
}


def norm_text(value) -> str:
    return re.sub(r"\s+", " ", str(value or "").strip())


def hitung_kata(text: str) -> int:
    return len(re.findall(r"\b\w+\b", str(text or "")))


def label_indonesia(label: str) -> str:
    label = str(label)
    if label == "OUT_OF_TOPIC":
        return "Di luar topik"
    parts = label.split("_")
    if len(parts) >= 2:
        aspek = ASPEK_ID.get(parts[0], parts[0].title())
        sentimen = SENTIMEN_ID.get(parts[1], parts[1].title())
        return f"{aspek} - {sentimen}"
    return label


def aspek_label(label: str) -> str:
    label = str(label)
    if label == "OUT_OF_TOPIC":
        return "Di luar topik"
    return ASPEK_ID.get(label.split("_")[0], label.split("_")[0].title())


def sentimen_label(label: str) -> str:
    label = str(label)
    if label == "OUT_OF_TOPIC":
        return "Di luar topik"
    parts = label.split("_")
    return SENTIMEN_ID.get(parts[1], parts[1].title()) if len(parts) > 1 else ""


def tampilkan_kolom_id(df: pd.DataFrame) -> pd.DataFrame:
    if df.empty:
        return df
    tampil = df.copy()
```

```
    return tampil.rename(columns={c: KOLOM_ID.get(c, c) for c in tampil.columns})


def cari_file(*nama_file: str) -> Path | None:
    for nama in nama_file:
        p = DATA_DIR / nama
        if p.exists():
            return p
    return None


def load_json_or_jsonl(path: Path):
    text = path.read_text(encoding="utf-8-sig").strip()
    if not text:
        return []
    # File IRR dari Prodigy sering berekstensi .jsonl, tetapi isinya satu objek JSON.
    if text[0] in "[{":
        try:
            parsed = json.loads(text)
            return parsed
        except json.JSONDecodeError:
            pass
    rows = []
    buffer = ""
    for line in text.splitlines():
        if not line.strip():
            continue
        buffer += line.strip()
        try:
            rows.append(json.loads(buffer))
            buffer = ""
        except json.JSONDecodeError:
            buffer += "\n"
    if buffer.strip():
        try:
            rows.append(json.loads(buffer))
        except json.JSONDecodeError as exc:
            st.warning(f"Ada bagian JSONL yang tidak bisa dibaca pada file {path.name}: {exc}")
    return rows


def load_jsonl_rows(path: Path) -> list[dict]:
    obj = load_json_or_jsonl(path)
    if isinstance(obj, list):
        return obj
    return []


def buat_agregasi(annotation_rows: list[dict], clean_df: pd.DataFrame, raw_df: pd.DataFrame) -> tuple[pd.DataFra
me, pd.DataFrame, pd.DataFrame]:
    raw_df = raw_df.copy()
    clean_df = clean_df.copy()
    if "text" in raw_df.columns:
        raw_df["text_norm"] = raw_df["text"].map(norm_text)
    else:
        raw_df["text_norm"] = ""
    if "text" in clean_df.columns:
        clean_df["text_norm"] = clean_df["text"].map(norm_text)
    else:
        clean_df["text_norm"] = ""

    raw_meta = {}
    if {"text_norm", "title"}.issubset(raw_df.columns):
        cols = [c for c in ["title", "stars"] if c in raw_df.columns]
        raw_meta = raw_df.dropna(subset=["text_norm"]).drop_duplicates("text_norm").set_index("text_norm")[cols]
        .to_dict("index")
    clean_meta = {}
    if {"text_norm", "title"}.issubset(clean_df.columns):
        clean_meta = clean_df.drop_duplicates("text_norm").set_index("text_norm")[["title"]].to_dict("index")

    by_text: dict[str, list[dict]] = defaultdict(list)
    for row in annotation_rows:
        by_text[norm_text(row.get("text", ""))].append(row)

    review_rows = []
    label_rows = []
    entity_rows = []

    for review_id, (text_norm, rows) in enumerate(by_text.items(), start=1):
        if not text_norm:
            continue
        text = rows[0].get("text", text_norm)
        annotators = sorted({r.get("_annotator_id", "") for r in rows if r.get("_annotator_id")})
        n_annotators = len(annotators) if annotators else len(rows)
        min_votes = 1 if n_annotators <= 1 else (n_annotators // 2 + 1)

        counts = Counter()
        for r in rows:
            for lab in r.get("accept") or []:
                counts[lab] += 1
        final_labels = [lab for lab in LABELS if counts.get(lab, 0) >= min_votes]
        if not final_labels and counts:
            max_vote = max(counts.values())
            final_labels = [lab for lab in LABELS if counts.get(lab, 0) == max_vote]

        meta = dict(clean_meta.get(text_norm, {}))
        meta.update(raw_meta.get(text_norm, {}))
        business_name = meta.get("title", "")
        rating = meta.get("stars", "")

        review_rows.append({
```

```
            "review_id": review_id,
            "category": "Kuliner",
            "business_name": business_name,
            "rating": rating,
            "review_text": text,
            "clean_text": text_norm,
            "word_count": hitung_kata(text),
            "token_count": len(rows[0].get("tokens") or []),
            "n_annotations": len(rows),
            "n_annotators": n_annotators,
            "final_labels": final_labels,
            "final_labels_id": ", ".join(label_indonesia(l) for l in final_labels),
        })
        for lab in final_labels:
            label_rows.append({
                "review_id": review_id,
                "text": text,
                "kode_label": lab,
                "label": label_indonesia(lab),
                "aspek": aspek_label(lab),
                "sentimen": sentimen_label(lab),
                "jumlah_annotator_setuju": counts.get(lab, 0),
                "jumlah_annotator": n_annotators,
            })
        for r in rows:
            for sp in r.get("spans") or []:
                start, end = sp.get("start"), sp.get("end")
                entitas = text[start:end] if isinstance(start, int) and isinstance(end, int) else sp.get("text",
             "")
                lab = sp.get("label", "")
                entity_rows.append({
                    "review_id": review_id,
                    "text": text,
                    "annotator_id": r.get("_annotator_id", ""),
                    "entitas": norm_text(entitas),
                    "kode_label": lab,
                    "label": label_indonesia(lab),
                    "aspek": aspek_label(lab),
                    "sentimen": sentimen_label(lab),
                })

    return pd.DataFrame(review_rows), pd.DataFrame(label_rows), pd.DataFrame(entity_rows)


@st.cache_data(show_spinner=False)
def load_data():
    raw_path = cari_file("Kelp1_dataset_1.csv")
    clean_path = cari_file("Kelp1_dataset_2.csv")
    ann_path = cari_file("db_nlp1_genap2526.jsonl", "Kelp1_dataset_anotasi.jsonl")

    if raw_path is None or clean_path is None or ann_path is None:
        missing = []
        if raw_path is None:
            missing.append("Kelp1_dataset_1.csv")
        if clean_path is None:
            missing.append("Kelp1_dataset_2.csv")
        if ann_path is None:
            missing.append("db_nlp1_genap2526.jsonl atau Kelp1_dataset_anotasi.jsonl")
        raise FileNotFoundError("File berikut belum ada di folder dataset/: " + ", ".join(missing))

    raw_df = pd.read_csv(raw_path)
    clean_df = pd.read_csv(clean_path)
    annotation_rows = load_jsonl_rows(ann_path)

    # Hitung ulang data agregasi dari anotasi penuh agar label dan NER selalu sinkron
    # dengan dataset terbaru. File aggregated tetap boleh ada sebagai cadangan, tetapi
    # sumber utama aplikasi adalah db_nlp1_genap2526.jsonl.
    reviews_df, labels_df, entities_df = buat_agregasi(annotation_rows, clean_df, raw_df)

    # Perbaikan tipe dan kolom agar tampilan stabil.
    for df in [reviews_df, labels_df, entities_df]:
        for col in df.columns:
            if df[col].dtype == "object":
                df[col] = df[col].fillna("")
    if "word_count" in reviews_df.columns:
        reviews_df["word_count"] = pd.to_numeric(reviews_df["word_count"], errors="coerce").fillna(0).astype(int
        )
    if "token_count" in reviews_df.columns:
        reviews_df["token_count"] = pd.to_numeric(reviews_df["token_count"], errors="coerce").fillna(0).astype(i
        nt)
    if "category" not in reviews_df.columns:
        reviews_df["category"] = "Kuliner"
    if "business_name" not in reviews_df.columns:
        reviews_df["business_name"] = ""
    if "rating" not in reviews_df.columns:
        reviews_df["rating"] = ""
    return raw_df, clean_df, reviews_df, labels_df, entities_df, annotation_rows


def filter_data(reviews_df: pd.DataFrame, labels_df: pd.DataFrame, entities_df: pd.DataFrame):
    st.sidebar.title("Explorer Kelp1")
    kategori_opsi = sorted([x for x in reviews_df.get("category", pd.Series(dtype=str)).dropna().unique() if str
    (x).strip()])
    kategori_pilih = st.sidebar.multiselect("Filter kategori usaha", kategori_opsi, default=kategori_opsi)

    aspek_opsi = sorted(labels_df["aspek"].dropna().unique().tolist()) if not labels_df.empty and "aspek" in lab
    els_df.columns else []
    sentimen_opsi = sorted(labels_df["sentimen"].dropna().unique().tolist()) if not labels_df.empty and "sentime
    n" in labels_df.columns else []
    aspek_pilih = st.sidebar.multiselect("Filter aspek", aspek_opsi, default=aspek_opsi)
```

```
    sentimen_pilih = st.sidebar.multiselect("Filter sentimen", sentimen_opsi, default=sentimen_opsi)
    pencarian = st.sidebar.text_input("Cari ulasan / nama usaha")

    filtered_reviews = reviews_df.copy()
    if kategori_pilih:
        filtered_reviews = filtered_reviews[filtered_reviews["category"].isin(kategori_pilih)]

    filtered_labels = labels_df.copy()
    if not filtered_labels.empty:
        if aspek_pilih:
            filtered_labels = filtered_labels[filtered_labels["aspek"].isin(aspek_pilih)]
        if sentimen_pilih:
            filtered_labels = filtered_labels[filtered_labels["sentimen"].isin(sentimen_pilih)]
        if aspek_pilih or sentimen_pilih:
            filtered_reviews = filtered_reviews[filtered_reviews["review_id"].isin(filtered_labels["review_id"].
            unique())]

    if pencarian.strip():
        q = pencarian.strip().lower()
        mask = (
            filtered_reviews["review_text"].astype(str).str.lower().str.contains(q, regex=False)
            | filtered_reviews["business_name"].astype(str).str.lower().str.contains(q, regex=False)
        )
        filtered_reviews = filtered_reviews[mask]

    filtered_labels = labels_df[labels_df["review_id"].isin(filtered_reviews["review_id"])] if not labels_df.emp
    ty else labels_df
    filtered_entities = entities_df[entities_df["review_id"].isin(filtered_reviews["review_id"])] if not entitie
    s_df.empty else entities_df
    return filtered_reviews, filtered_labels, filtered_entities


def bar_chart_count(df: pd.DataFrame, col: str, title: str):
    st.subheader(title)
    if df.empty or col not in df.columns:
        st.info("Data belum tersedia.")
        return
    counts = df[col].value_counts().rename_axis(col).reset_index(name="Jumlah")
    st.bar_chart(counts.set_index(col))
    with st.expander("Lihat tabel jumlah"):
        st.dataframe(tampilkan_kolom_id(counts), use_container_width=True, hide_index=True)


def halaman_ringkasan(raw_df, clean_df, reviews_df, labels_df, entities_df, annotation_rows):
    st.title("Ringkasan Dataset ABSA Kelompok 1")
    st.write("Aplikasi ini menggunakan dataset mentah, dataset hasil preprocessing, data anotasi Prodigy, serta
    file IRR dari dosen/kelompok.")

    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Data mentah", f"{len(raw_df):,}")
    c2.metric("Data hasil preprocessing", f"{len(clean_df):,}")
    c3.metric("Review teranotasi unik", f"{len(reviews_df):,}")
    annotators = sorted({r.get("_annotator_id", "") for r in annotation_rows if r.get("_annotator_id")})
    c4.metric("Jumlah annotator", f"{len(annotators):,}")

    c5, c6, c7 = st.columns(3)
    c5.metric("Rata-rata jumlah kata", f"{reviews_df['word_count'].mean():.2f}" if not reviews_df.empty else "0"
    )
    c6.metric("Median jumlah kata", f"{reviews_df['word_count'].median():.0f}" if not reviews_df.empty else "0")
    c7.metric("Jumlah label final", f"{len(labels_df):,}")

    col_a, col_b = st.columns(2)
    with col_a:
        bar_chart_count(labels_df, "label", "Distribusi Label ABSA Final")
    with col_b:
        bar_chart_count(labels_df, "aspek", "Distribusi Data per Aspek")

    st.subheader("Contoh Data Review")
    cols = ["review_id", "category", "business_name", "rating", "review_text", "word_count", "final_labels_id"]
    st.dataframe(tampilkan_kolom_id(reviews_df[cols].head(20)), use_container_width=True, hide_index=True)


def halaman_telusuri(reviews_df, labels_df, entities_df):
    st.title("Telusuri Ulasan")
    st.write("Gunakan filter di sidebar untuk membatasi kategori, aspek, sentimen, dan kata kunci ulasan.")
    cols = ["review_id", "category", "business_name", "rating", "review_text", "word_count", "token_count", "fin
    al_labels_id"]
    st.dataframe(tampilkan_kolom_id(reviews_df[cols]), use_container_width=True, hide_index=True, height=520)

    st.subheader("Detail Ulasan")
    if reviews_df.empty:
        st.info("Tidak ada data sesuai filter.")
        return
    pilihan = st.selectbox("Pilih ID ulasan", reviews_df["review_id"].tolist())
    row = reviews_df[reviews_df["review_id"] == pilihan].iloc[0]
    st.markdown(f"**Nama usaha:** {row.get('business_name', '')}")
    st.markdown(f"**Rating:** {row.get('rating', '')}")
    st.markdown(f"**Label final:** {row.get('final_labels_id', '')}")
    st.write(row.get("review_text", ""))

    col1, col2 = st.columns(2)
    with col1:
        st.write("**Label ABSA ulasan ini**")
        tmp = labels_df[labels_df["review_id"] == pilihan]
        st.dataframe(tampilkan_kolom_id(tmp), use_container_width=True, hide_index=True)
    with col2:
        st.write("**Entitas NER ulasan ini**")
        tmp = entities_df[entities_df["review_id"] == pilihan]
        st.dataframe(tampilkan_kolom_id(tmp), use_container_width=True, hide_index=True)

def halaman_panel(labels_df, entities_df):
```

```
    st.title("Panel ABSA dan NER")
    tab1, tab2 = st.tabs(["Label ABSA", "Entitas NER"])
    with tab1:
        st.write("Tabel ini berisi label ABSA final hasil agregasi/voting mayoritas.")
        st.dataframe(tampilkan_kolom_id(labels_df), use_container_width=True, hide_index=True, height=560)
    with tab2:
        st.write("Tabel ini berisi entitas NER/spans yang terhubung dengan aspek dan sentimen.")
        st.dataframe(tampilkan_kolom_id(entities_df), use_container_width=True, hide_index=True, height=560)


def halaman_statistik(reviews_df, labels_df, entities_df):
    st.title("Statistik dan EDA")
    st.subheader("Distribusi Panjang Review")
    if not reviews_df.empty and "word_count" in reviews_df.columns:
        fig, ax = plt.subplots(figsize=(10, 4))
        ax.hist(reviews_df["word_count"].dropna(), bins=30)
        ax.set_xlabel("Jumlah kata")
        ax.set_ylabel("Jumlah review")
        ax.set_title("Distribusi Panjang Review Berdasarkan Jumlah Kata")
        st.pyplot(fig, clear_figure=True)
    else:
        st.info("Data panjang review belum tersedia.")

    col1, col2 = st.columns(2)
    with col1:
        bar_chart_count(labels_df, "label", "Distribusi Label ABSA")
    with col2:
        bar_chart_count(entities_df, "label", "Distribusi Entitas NER")

    st.subheader("Matriks Korelasi Antar Label")
    if labels_df.empty:
        st.info("Label belum tersedia.")
        return
    one_hot = pd.crosstab(labels_df["review_id"], labels_df["kode_label"])
    one_hot = one_hot.reindex(columns=[l for l in LABELS if l in one_hot.columns], fill_value=0)
    corr = one_hot.corr()
    if corr.empty:
        st.info("Matriks korelasi belum dapat dibuat.")
        return
    fig, ax = plt.subplots(figsize=(10, 8))
    im = ax.imshow(corr.values, aspect="auto")
    ax.set_xticks(range(len(corr.columns)))
    ax.set_yticks(range(len(corr.index)))
    ax.set_xticklabels([label_indonesia(c) for c in corr.columns], rotation=90)
    ax.set_yticklabels([label_indonesia(c) for c in corr.index])
    fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
    ax.set_title("Korelasi Antar Label ABSA")
    st.pyplot(fig, clear_figure=True)
    with st.expander("Lihat nilai korelasi"):
        corr_display = corr.copy()
        corr_display.index = [label_indonesia(i) for i in corr_display.index]
        corr_display.columns = [label_indonesia(c) for c in corr_display.columns]
        st.dataframe(corr_display, use_container_width=True)


def render_irr_table(obj, jenis: str):
    if not obj:
        st.info(f"Data IRR {jenis} belum tersedia.")
        return
    if isinstance(obj, dict) and all(isinstance(v, dict) for v in obj.values()):
        rows = []
        for label, metrics in obj.items():
            row = {"Kode Label": label, "Label": label_indonesia(label)}
            row.update(metrics)
            rows.append(row)
        df = pd.DataFrame(rows)
        st.dataframe(df, use_container_width=True, hide_index=True)
    elif isinstance(obj, dict):
        metric_keys = [k for k, v in obj.items() if not isinstance(v, (dict, list))]
        metric_df = pd.DataFrame([{"Metrik": k, "Nilai": obj[k]} for k in metric_keys])
        st.dataframe(metric_df, use_container_width=True, hide_index=True)
        if isinstance(obj.get("metrics_per_label"), dict):
            rows = []
            for label, metrics in obj["metrics_per_label"].items():
                row = {"Kode Label": label, "Label": label_indonesia(label)}
                row.update(metrics)
                rows.append(row)
            st.write("**Metrik per label**")
            st.dataframe(pd.DataFrame(rows), use_container_width=True, hide_index=True)
    else:
        st.json(obj)


def halaman_irr():
    st.title("Inter-Annotator Agreement (IRR)")
    st.write("Halaman ini menampilkan hasil IRR dari file yang sudah disediakan dosen/kelompok.")

    textcat_path = cari_file("nlp1_iaaa_textcat.jsonl", "Kelp1_irr_textcat.json", "Kelp1_irr_textcat.jsonl")
    ner_path = cari_file("nlp1_iaaa.jsonl", "Kelp1_irr_ner.json", "Kelp1_irr_ner.jsonl")

    st.subheader("IRR Klasifikasi Teks / Multi-label")
    if textcat_path is None:
        st.warning("File IRR textcat belum ditemukan. Letakkan `nlp1_iaaa_textcat.jsonl` di folder dataset/.")
    else:
        st.caption(f"File yang dibaca: {textcat_path.name}")
        render_irr_table(load_json_or_jsonl(textcat_path), "textcat")

    st.subheader("IRR NER / Span")
    if ner_path is None:
        st.warning("File IRR NER belum ditemukan. Letakkan `nlp1_iaaa.jsonl` di folder dataset/.")
```

```
    else:
        st.caption(f"File yang dibaca: {ner_path.name}")
        render_irr_table(load_json_or_jsonl(ner_path), "NER")


def main():
    try:
        raw_df, clean_df, reviews_df, labels_df, entities_df, annotation_rows = load_data()
    except Exception as exc:
        st.error("Aplikasi gagal memuat data.")
        st.exception(exc)
        return

    filtered_reviews, filtered_labels, filtered_entities = filter_data(reviews_df, labels_df, entities_df)

    st.sidebar.divider()
    st.sidebar.metric("Data terfilter", f"{len(filtered_reviews):,}")
    st.sidebar.metric("Total ulasan", f"{len(reviews_df):,}")

    halaman = st.sidebar.radio(
        "Halaman",
        ["Ringkasan", "Telusuri Ulasan", "Panel ABSA dan NER", "Statistik", "IRR"],
    )

    if halaman == "Ringkasan":
        halaman_ringkasan(raw_df, clean_df, filtered_reviews, filtered_labels, filtered_entities, annotation_row
        s)
    elif halaman == "Telusuri Ulasan":
        halaman_telusuri(filtered_reviews, filtered_labels, filtered_entities)
    elif halaman == "Panel ABSA dan NER":
        halaman_panel(filtered_labels, filtered_entities)
    elif halaman == "Statistik":
        halaman_statistik(filtered_reviews, filtered_labels, filtered_entities)
    else:
        halaman_irr()


if __name__ == "__main__":
    main()
```